Lập trình mạng · Chương 3

UDP: datagram và DatagramSocket

TS. Đặng Ngọc Hùng

Chương 3: Lập trình UDP và giao thức ứng dụng hiện đại

1 UDP

Datagram và DatagramSocket

2 Multicast

Nhóm multicast, network interface và TTL

Bài toán ứng dụng: truyền video theo nhóm multicast

3 WebSocket

Nâng cấp từ HTTP và giao tiếp hai chiều

Bài toán ứng dụng: chat thời gian thực bằng WebSocket

4 HTTP/2

Binary framing và multiplexing

5 HTTP/3

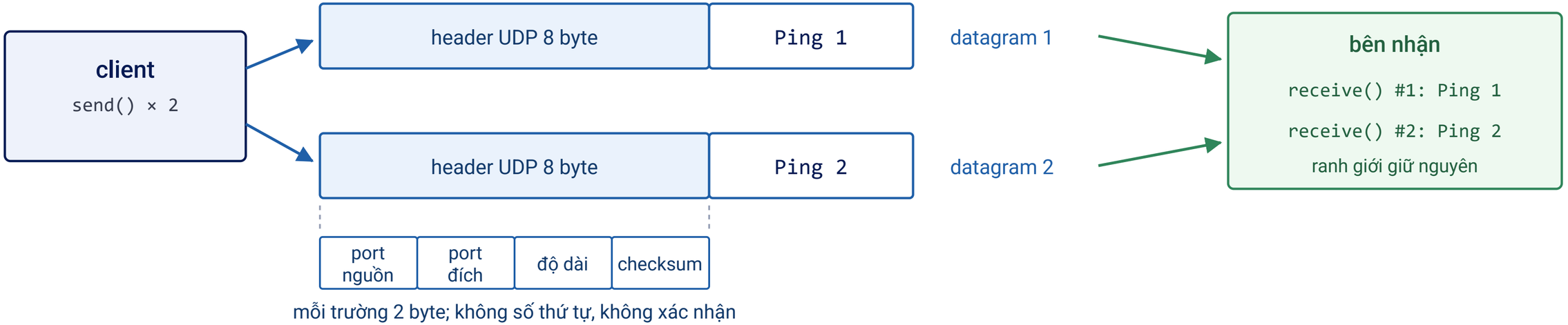
QUIC và giao thức trên UDP

UDP gửi ngay một datagram, không cần kết nối

1. Đo độ trễ, hỏi giờ, truy vấn DNS: chỉ **một gói nhỏ** đi và về, mà TCP cần **bắt tay ba bước** trước
2. UDP: gửi ngay một **datagram**, không kết nối; đổi lại không bảo đảm datagram đến nơi
3. Không có kết nối: bên nhận phải tự quản lý **buffer** và **thời gian chờ**

> **Câu hỏi của phần:** gửi và nhận datagram bằng Java thế nào, và cần chú ý gì vì UDP không có kết nối?

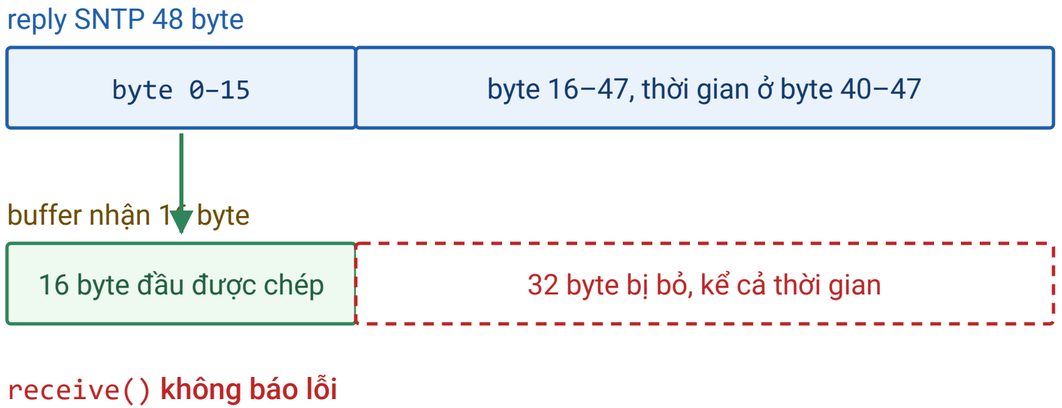
Mỗi lần send là một datagram độc lập, giữ nguyên ranh giới



Datagram: mỗi `send()` một datagram, mỗi `receive()` một datagram; header UDP 8 byte, không số thứ tự, không xác nhận.

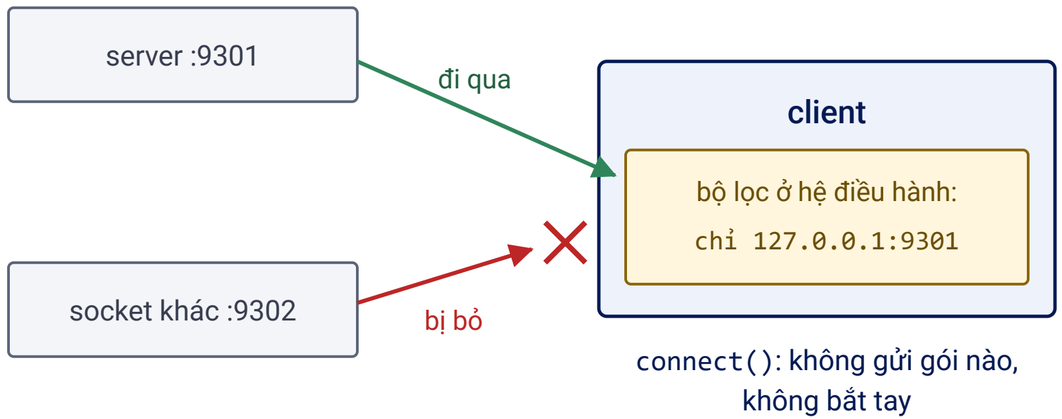
Datagram dài hơn buffer bị cắt, connected UDP socket chỉ lọc endpoint

Cắt ngắn datagram



`receive()` **không báo lỗi**

Connected UDP socket



`connect()`: không gửi gói nào, không bắt tay

> **Cắt ngắn datagram**: phần dư bị bỏ, không báo lỗi. **Connected UDP socket**: `connect()` chỉ đặt bộ lọc ở hệ điều hành, không gửi gói nào.

Server ping chỉ có một socket, nhận một datagram và trả một datagram

Java

```
try (DatagramSocket socket = new DatagramSocket(12000)) {  // UDP port 12000: không có listening socket
    while (true) {
        DatagramPacket request = new DatagramPacket(buf, buf.length);
        socket.receive(request);  // chờ và nhận đúng một datagram
        String message = new String(request.getData(), 0, request.getLength(), UTF_8);
        if (random.nextInt(11) < 4) {  // mô phỏng mất datagram, như bản gốc của bài
            continue;
        }
        byte[] reply = message.toUpperCase().getBytes(UTF_8);
        socket.send(new DatagramPacket(reply, reply.length, request.getSocketAddress()));  // trả về nơi gửi
```

Kết quả chạy

```
UDP ping server on port 12000
from /127.0.0.1:52239: Ping 1 09:49:05.743 -> dropped
from /127.0.0.1:52239: Ping 2 09:49:06.774 -> replied
from /127.0.0.1:52239: Ping 3 09:49:06.800 -> replied
from /127.0.0.1:52239: Ping 4 09:49:06.801 -> dropped
from /127.0.0.1:52239: Ping 5 09:49:07.809 -> replied
from /127.0.0.1:52239: Ping 6 09:49:07.812 -> replied
from /127.0.0.1:52239: Ping 7 09:49:07.814 -> replied
from /127.0.0.1:52239: Ping 8 09:49:07.816 -> replied
from /127.0.0.1:52239: Ping 9 09:49:07.817 -> replied
from /127.0.0.1:52239: Ping 10 09:49:07.818 -> replied
```

Client ping gửi ngay từng datagram và tự giới hạn thời gian chờ phản hồi

Java

```
socket.setSoTimeout(1_000);  // UDP không báo mất datagram: client tự giới hạn thời gian chờ
for (int seq = 1; seq <= 10; seq++) {
    byte[] data = ("Ping " + seq + " " + LocalTime.now().format(TIME)).getBytes(UTF_8);
    long start = System.nanoTime();
    socket.send(new DatagramPacket(data, data.length, server));  // gửi ngay, không bắt tay
    try {
        socket.receive(reply);  // nhận một datagram
        double rtt = (System.nanoTime() - start) / 1e6;
    } catch (SocketTimeoutException e) {
        System.out.println("Ping " + seq + ": request timed out");
```

Kết quả chạy

```
Ping 1: request timed out
PING 2 09:49:06.774, RTT 1.94 ms
PING 3 09:49:06.800, RTT 0.46 ms
Ping 4: request timed out
PING 5 09:49:07.809, RTT 1.18 ms
PING 6 09:49:07.812, RTT 1.30 ms
PING 7 09:49:07.814, RTT 1.12 ms
PING 8 09:49:07.816, RTT 1.10 ms
PING 9 09:49:07.817, RTT 0.30 ms
PING 10 09:49:07.818, RTT 0.25 ms
```

connect() trên DatagramSocket trả về ngay và chỉ lọc endpoint

Java

```
probe.connect(new InetSocketAddress("192.0.2.1", 9199));  // không gửi gì, không chờ phản hồi
client.connect(new InetSocketAddress("127.0.0.1", 9301));  // chỉ nhận từ endpoint của server
send(other, "from other endpoint", client);  // gửi tới client từ port 9302
send(server, "from server", client);  // gửi tới client từ port 9301
```

Kết quả chạy

```
connect() to 192.0.2.1:9199 returned after 30 ms, isConnected = true
receive #1: "from server" from port 9301
receive #2: nothing more (datagram from port 9302 was dropped)
```

Buffer nhận phải đủ cho datagram của giao thức, vì receive() không báo khi cắt

Java

```
byte[] request = new byte[NTP_PACKET];
request[0] = 0x1B;  // LI = 0, phiên bản 3, chế độ 3: client (RFC 4330)
socket.send(new DatagramPacket(request, request.length, new InetSocketAddress(host, 123)));
DatagramPacket reply = new DatagramPacket(new byte[bufferSize], bufferSize);
socket.receive(reply);  // dài hơn buffer thì bị cắt, không báo lỗi
if (reply.getLength() < NTP_PACKET) {  // chỉ biết thiếu nhờ đặc tả ghi rõ 48 byte
    System.out.println(", no Transmit Timestamp (bytes 40-47)");
    return;
}
long seconds = ByteBuffer.wrap(reply.getData(), 40, 4).getInt() & 0xFFFF_FFFFL;  // giây kể từ 1900
```

Kết quả chạy

```
buffer 48: received 48 bytes, server time 2026-10-26T02:49:38Z
buffer 16: received 16 bytes, no Transmit Timestamp (bytes 40-47)
```

- Reply 48 byte, buffer 16 byte: nhận **16 byte**, không có ngoại lệ; thời gian ở byte 40–47 **bị mất**
- Đặc tả phải ghi **độ dài** của datagram, như SNTP ghi 48 byte; buffer nhận lấy theo con số đó

Câu hỏi 1

Một datagram dài 40 byte đến, nhưng DatagramPacket chỉ có buffer 16 byte. Lời gọi `receive()` làm gì?

A. Ném ngoại lệ báo buffer quá nhỏ.

B. Nhận 16 byte đầu, bỏ 24 byte còn lại, không báo lỗi.

C. Nhận 16 byte, giữ 24 byte còn lại cho lần `receive()` sau.

D. Chờ đến khi chương trình cấp buffer đủ lớn.

Câu hỏi 1

Một datagram dài 40 byte đến, nhưng DatagramPacket chỉ có buffer 16 byte. Lời gọi `receive()` làm gì?

A. Ném ngoại lệ báo buffer quá nhỏ.

B. Nhận 16 byte đầu, bỏ 24 byte còn lại, không báo lỗi.

C. Nhận 16 byte, giữ 24 byte còn lại cho lần `receive()` sau.

D. Chờ đến khi chương trình cấp buffer đủ lớn.

Mỗi lời gọi `receive()` nhận trọn một datagram; phần không vừa buffer bị bỏ và không có ngoại lệ. Giữ phần còn lại cho lần đọc sau là hành vi của byte stream TCP.

Câu hỏi 2

Client gọi `send()` hai lần liên tiếp, với Ping 1 rồi Ping 2. Ở bên nhận, điều nào có thể xảy ra?

A. Một lần `receive()` nhận được Ping 1Ping 2.

B. UDP tự ghép hai datagram để tiết kiệm header.

C. Hai lần `receive()`, mỗi lần đúng một datagram; một datagram cũng có thể bị mất hoặc đến sau datagram kia.

D. Lần `receive()` thứ hai luôn nhận được Ping 2 ngay sau Ping 1, đúng thứ tự gửi.

Câu hỏi 2

Client gọi `send()` hai lần liên tiếp, với Ping 1 rồi Ping 2. Ở bên nhận, điều nào có thể xảy ra?

A. Một lần `receive()` nhận được Ping 1Ping 2.

B. UDP tự ghép hai datagram để tiết kiệm header.

C. Hai lần `receive()`, mỗi lần đúng một datagram; một datagram cũng có thể bị mất hoặc đến sau datagram kia.

D. Lần `receive()` thứ hai luôn nhận được Ping 2 ngay sau Ping 1, đúng thứ tự gửi.

Datagram giữ ranh giới: mỗi `receive()` nhận đúng một datagram, không bao giờ ghép. UDP không bảo đảm đến nơi hay đúng thứ tự, nên một datagram có thể mất hoặc đến sau.

Tổng kết

1. **Datagram**: mỗi `send()` một datagram, mỗi `receive()` một datagram; header UDP 8 byte; không bảo đảm đến nơi
2. Không có kết nối: client tự đặt **read timeout**; buffer phải đủ, vì **cắt ngắn datagram** không báo lỗi
3. **Connected UDP socket**: `connect()` chỉ lọc endpoint ở hệ điều hành, không bắt tay

Từ khóa cần nhớ datagram · DatagramSocket · DatagramPacket · cắt ngắn datagram · connected UDP socket

Mở rộng

Chủ đề nâng cao

- Kích thước datagram an toàn và MTU
- Độ tin cậy ở tầng ứng dụng trên UDP (Chương 5)

Bài tập mở rộng

Bổ sung cho client ping phần thống kê như lệnh ping: RTT nhỏ nhất, lớn nhất, trung bình và tỉ lệ mất gói (bài tập tùy chọn của UDP Pinger).

Tiếp theo

Multicast: nhóm multicast, network interface và TTL